มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วน โดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 4 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 4 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.

Project Phase 2 From Design to Conforming Code

ITCS382 SOFTWARE DESIGN AND MODELING

GEMINI PROJECT



Project Phase 2 at a Glance

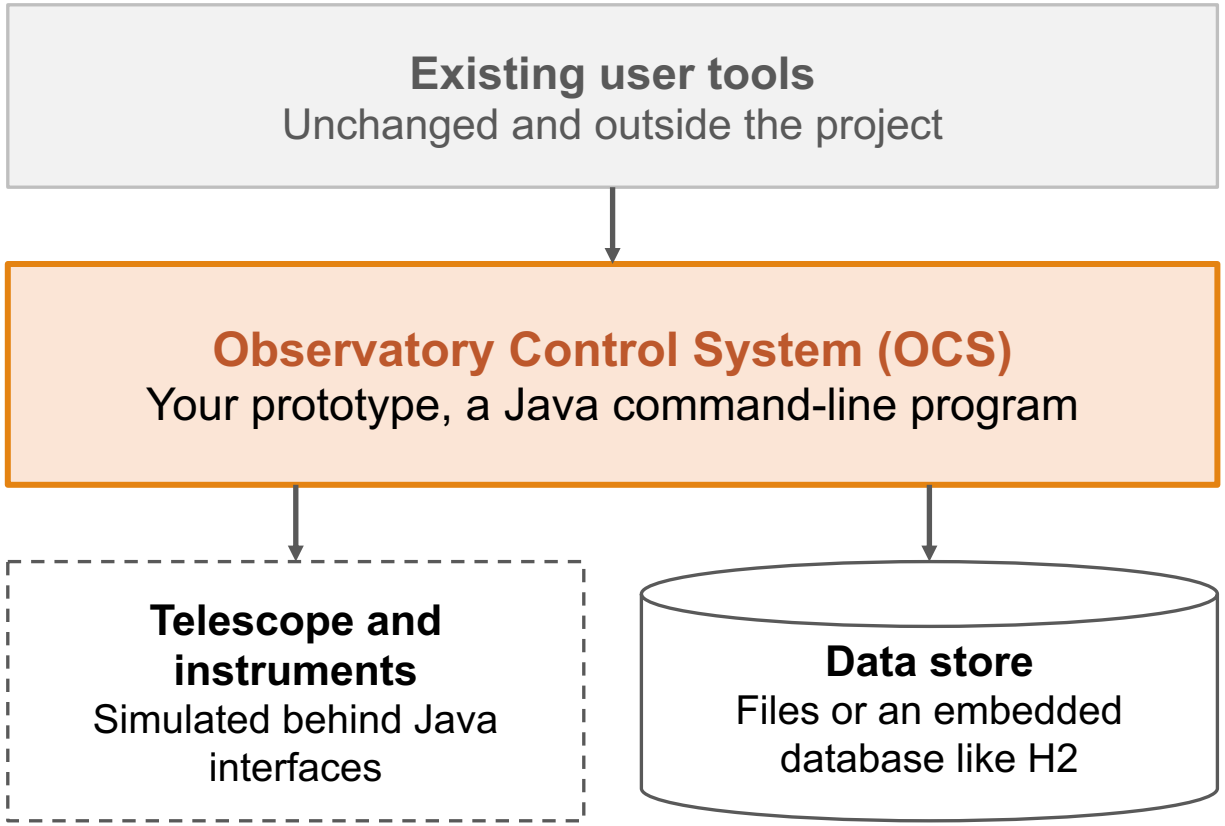
Item	Detail
Deliverable	D4, a design model, a working prototype and a report
Use cases	Four use cases selected from your Phase 1 analysis
Weight	12.5% of the total course score
Deadline	Saturday 14 November 2026, 23:55 (commit time)
Presentation	Monday 16 November 2026, 08:45 to 12:15

The Focus of This Phase

- Refine your Phase 1 analysis into a design model that is detailed enough to guide implementation.
- Implement four use cases in Java so that the code conforms to this design.
- Apply cohesion, coupling, connascence, design patterns and architectural views to your own design of the Gemini system.

The marks follow the quality of your design and how closely your code conforms to it. Features beyond the four use cases earn no marks.

What You Build

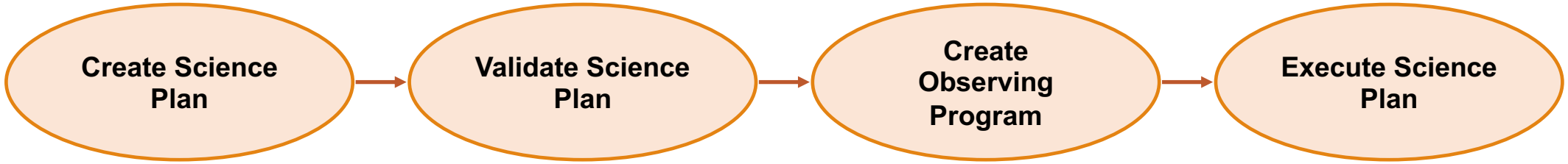


Constraints

- Java 17 or later, built with Maven
- No web interface or application framework
- One external library, for the data store only
- No separate database server
- Creates its own files or tables on first run
- Builds with mvn package from a fresh clone

Selecting Your Four Use Cases

- Select four use cases from your Phase 1 analysis.
- They must share classes and data and form one connected flow.
- At least one use case involves both a science plan and an observing program.
- Login does not count. Each sequence diagram involves at least three of your own classes.
- Name the four use cases in your design file and justify the choice.



Each use case works on the science plan or observing program that the previous one produced.

Deliverable D4 in Your Repository

Item at the repository root	Part	Contents
<groupname>_D4_Design.md	Part 1, design model	Package diagram, class diagram, four sequence diagrams, four use case descriptions
ocs folder	Part 2, prototype	The Maven project, built with mvn package
<groupname>_D4_Report.md	Part 3, report	Design rationale, run instructions, use case owners, traceability table, SonarQube and AI review results, AI usage declaration
reviews folder	Evidence	The complete output of every AI review run

One submission, due Saturday 14 November 2026 at 23:55 (commit time).

Part 1: Package Diagram

Package diagram

- The packages of the OCS and the classes in each package
- Each package in the diagram is one Java package
- Arrows for the only dependencies your code may have
- The interfaces that hide external systems, their simulated versions, and the package that uses the data store

Write every diagram in Mermaid, and use an image only when Mermaid cannot draw the package diagram.

Part 1: Class and Sequence Diagrams

Class diagram

- Visibility and type of every attribute
- Operations with parameter types and a return type
- Multiplicity, navigability and role names
- Collection types, for example List<Observation>
- Constructors, interfaces and abstract classes
- The classes of your design pattern marked

Sequence diagram for each use case

- Lifelines that are objects of your classes
- Messages that are operations with arguments
- Return messages that show returned values
- Object creation where it happens
- alt, opt and loop where the code branches or repeats
- A start at the actor or the command-line class

Revise the four use case descriptions from Phase 1 so that they match the design.

Part 2: Code That Follows the Design

1. Every class, attribute and operation appears in the code with the same name and type.
2. Visibility matches, so a minus sign becomes private and a plus sign becomes public.
3. Multiplicity matches, so 1 or 0..1 becomes one field and * becomes a List or another collection.
4. Navigability matches, so only the class at the tail of an arrow holds the reference.
5. Underlined attributes are static, and attributes marked / are computed rather than stored.
6. Every message in a sequence diagram is a call on the correct object, in the same order.
7. Nothing extra appears, such as a public field or a setter that the design does not have.

Each class sits in its package from the package diagram. When the code cannot follow the design, revise the design first, then the code.

Part 3: The Report

Design rationale

- How the design achieves high cohesion and low coupling, and keeps connascence weak
- The choice of data store, and how the rest of the code stays independent of it
- At least one design pattern, the problem it solves, its roles, and why it beats a simpler design

Evidence

- Run commands for the main flow and one alternate flow of each use case
- The owner of each use case
- A traceability table from use case step to message to method
- SonarQube results, with a fix or a reason for each issue
- AI review results, with model, commit and responses
- An AI usage declaration

Two Quality Checks

SonarQube

- General code quality, such as code smells, probable bugs, complex methods and duplicated code
- Install SonarQube for IDE and fix issues as you code
- We run it on your submitted commit for grading

AI review prompt

- In Appendix A of the Project 2 Description
- Compares the diagrams with the code
- Checks cohesion, coupling and connascence in design and code
- Record the exact model and version for every run
- Results do not directly affect your score

Text in your repository that tries to instruct an AI reviewer counts as academic misconduct.

The Phase 2 Presentation

- Monday 16 November 2026, from 08:45 to 12:15, before and after the regular class time
- 15 minutes per team, in slots assigned by the instructor
- All members attend and take part

The presentation covers

- Your package diagram, class diagram and design pattern, with the reason for the pattern
- A demonstration of the four use cases from the command line
- A trace of one message that we pick, in which the use case owner shows the line of code that sends it

Grading Criteria

Criterion	Weight	What is graded
Design-level models are correct	25%	Notation, types, visibility, multiplicity, navigation, signatures, fragments
Models agree with each other	15%	Lifelines, messages, use case steps and packages consistent across the diagrams
Code follows the design	**30%**	**The seven rules, packages that match the package diagram, the message trace**
Design quality	20%	Cohesion, coupling, connascence and encapsulation in the design, and a design pattern that solves a real problem
Code quality	10%	Cohesion, coupling and connascence in the code, and SonarQube results with every issue fixed or explained

AI Usage at Level 3, AI Collaboration

- You may use AI for ideas, drafting, feedback and refinement.
- Critically evaluate and modify what the AI suggests.
- Each member explains the design and code of their own use case without help.
- Declare every use of AI, with the exact model, in your report.
- Declared use within Level 3 does not lower your score.

Key Dates and Next Steps

- Read the Project 2 Description and the additional information documents on MyCourses.
- Choose your four use cases and agree on one owner for each.
- Ask the product owner, through the same GitHub issue, when the domain is unclear.
- Submit by Saturday 14 November 2026 at 23:55 (commit time).
- Present on Monday 16 November 2026, from 08:45 to 12:15.